

1 Authorization

The authorization mechanism is used to protect functions such as applications or buttons in the toolbar against unauthorized use and/or hide fields or field groups in the GUI or prevent them from being edited.

In order to protect functions or fields, you can assign an **authorization key**. Once assigned, you must explicitly **activate** the key if you want to use the functions or fields.

The following chapters describe the assignment of authorization keys to functions and fields and the assignment of authorization keys to function authorizations, licenses and feature sets.

1.1 Activate authorization keys

1.1.1 How it works

You can use three components to activate an authorization key. The authorization is therefore based on "three pillars". For the MES Development Suite, only the *Function authorization* is relevant.

Authorization key		
Function authorization	License	Feature set
Assignment to one or several function authorization(s)	Assignment to one or several licenses (mostly used by MPDV only)	Assignment to one or several feature sets (mostly used by MPDV only)

This chapter does not only treat the function authorizations, but also briefly considers the two other components in order to gain an overall understanding of the processes.

An authorization key is activated

- if it is assigned to at least one of the three components
- if the requirements of all assigned components are met

1.1.2 Function authorizations

You use the files "functionauthorisation.txt" to assign function authorizations to the authorization keys in the MOC.

If several function authorizations are assigned to one authorization key, the logged on user must have assigned at least one of the function authorizations.

Example:

...\resources\data\authorizations\FunctionAuthorisationMappings\functionauthorisation.txt

```
...  
pcccfg;pcccfg  
wpas.sug;wpas.sug  
grapt.spe;grapt.spe  
chkresp;CheckResponsibilityareaSelection  
doctype;doctype  
mdres.resmgmt;mdres.resmgmt  
uslo;uslo  
pfautp;pfautp  
pfunk;pfunk  
autcockpit;autcockpit  
mslmonitoring;mslmonitoring  
...
```

The first column includes the function authorization. The authorization key is after the semicolon.



Always create your own assignment files in the "local" scope!

1.1.3 Licenses

The assignment of authorization keys to licenses is only used by MPDV. It is realized using a "functionMapping". The mapping files are stored on the server in JDIR or JHYDRADIR.

...\jhydradir\MOC\1\functionMapping\standard*.lic

...\jdir\MOC\1\functionMapping\standard*.lic

The files are generated by a tool and protected against changes by a checksum. If several licenses are assigned to one authorization key, at least one of the assigned licenses must be activated.

1.1.4 Feature sets

The assignment of authorization keys to feature sets is only used by MPDV.

You can use feature sets to activate specific functions in accordance with a product version. Each FeatureSet is stored on the server in JDIR or JHYDRADIR.

...\jdir\MOC\1\configManager\standard*.xml

...\\jhydradir\MOC\1\configManager\standard*.xml

If several feature sets are assigned to one authorization key, at least one of the assigned feature sets must be fulfilled.

1.2 Authorization of functions and fields

You assign authorization keys to functions, applications and fields in the MOC using the "*.Authorization.xml" files in the folder "resources\data\authorizations". There is one file for each domain. The files in the "Standard" scope are secured against manipulation by a checksum. If a file is identified as being manipulated, all authorizations will fail.

Authorizations are maintained in the MDS repository, i.e. the above files are created by the export function of the Repository Client for MPDV modifications or can be generated/modified manually by customers for individual modifications.

1.2.1 Authorization of Fields

In order to authorize individual fields, the domain and the AuthorizationID with the acronym to be authorized must be maintained as a minimum. AuthorizationType must be "Acronym", AuthorizationKey must comprise the authorization key. If a service is entered in AuthorizationContext, the field must only be authorized in this service; if the column remains empty, authorization is always given.

Example:

```
<Authorization Domain="BOPerson"
  AuthorizationContext="BOPerson.list"
  AuthorizationId="person.premium_indicator"
  AuthorizationType="Acronym"
  AuthorizationKey="prgrp"
  AuthorizationDesignation="Extracted from code" />
```

1.2.2 Authorization of Field Groups

Important columns for the authorization of groups are AuthorizationContext and AuthorizationID. AuthorizationType must be "AcronymGroups", AuthorizationKey must comprise the authorization key. The AuthorizationID corresponds to the group name and usually is a language key. The AuthorizationContext is comprised of ApplicationID and Plugin-ID, separated by points.

Example: Group in layout panel of main application "Persons":

```

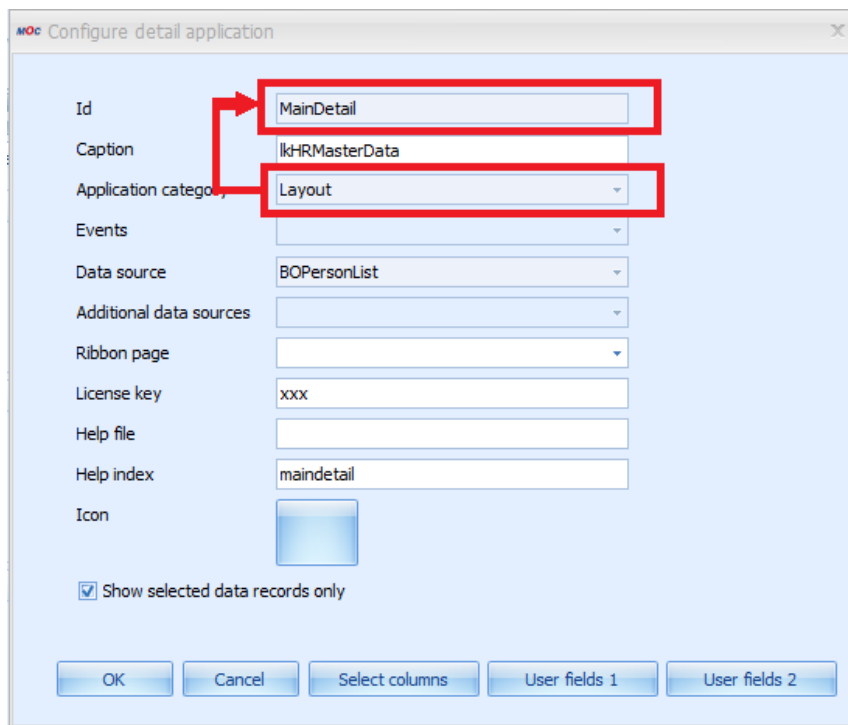
<Authorization Domain="BOPerson"
                AuthorizationContext="Persons.LayoutMainDetail"
                AuthorizationId="lkName"
                AuthorizationType="AcronymGroups"
                AuthorizationKey="testkey"
                AuthorizationDesignation="Extracted from code" />

```

Explanation for AuthorizationContext:

"Persons" is the application ID.

The plug-in ID "LayoutMainDetail" is composed of the application category and the detail application ID. This can be read in "Configure detailed applications":



Example: Group in editing dialog under "Persons":

```

<Authorization Domain="BOPerson"
                AuthorizationContext="Update.ParameterLayoutBOPersonUpdate"
                AuthorizationId="lkPersonLLE"
                AuthorizationType="AcronymGroups"
                AuthorizationKey="testkey"
                AuthorizationDesignation="Extracted from code" />

```

1.2.3 Authorization of applications



The authorization of (standard) applications is provided by whitelist management, i.e. applications are always protected and must be enabled by an authorization key. An application without authorization key can therefore not be used.

Applications not included in the authorization configuration cannot be opened. Authorization will also have an effect on whether or not a menu entry is activated.

AuthorizationID is the application name, AuthorizationType is "Application". The AuthorizationContext may be left empty.

Example:

```
<Authorization Domain="UnsortedApplications"
  AuthorizationContext=""
  AuthorizationId="Persons"
  AuthorizationType="Application"
  AuthorizationKey="pers.★"
  AuthorizationDesignation="Imported form menu" />
```

1.2.4 Authorization of Detail Applications

An authorization key is assigned in the detail application configuration.

The screenshot shows a Windows-style dialog box titled "Configure detail application". It contains several configuration fields:

- Id:** MainDetail
- Caption:** lkHRMasterData
- Application category:** Layout (dropdown)
- Events:** (empty dropdown)
- Data source:** BOPersonList (dropdown)
- Additional data sources:** (empty dropdown)
- Ribbon page:** (empty dropdown)
- License key:** xxx (this field is highlighted with a black rectangle)
- Help file:** (empty text box)
- Help index:** maindetail
- Icon:** (blue square icon)
- ☒ Show selected data records only

At the bottom, there are five buttons: OK, Cancel, Select columns, User fields 1, and User fields 2.

1.2.5 Authorization of Functions

Functions are authorized by assignment of an authorization key in the relevant CommandLink:

1.2.6 Authorization of ReferenceData entries

You can authorize ReferenceData entries by creating an authorization entry of type "RefDatItem". The context corresponds to the ReferenceData type. The value (db_key) is the authorization ID.

Example:

```
<Authorization Domain="BOPMaintenance"
  AuthorizationContext="rmcal.type"
  AuthorizationId="B"
  AuthorizationType="RefDatItem"
  AuthorizationKey="rmcaltypes"
  AuthorizationDesignation="Authorization for ReferenceData-item rmcal.type:B" />
```

Domain Set	Domain	ref_data_key	type	db_key	Description	is_default	designation	sort_key	Parent
Local	WageTypeInteractions	PersonnelConfigurationSelection:NONE	PersonnelConfigurationSelection	NONE		Y	lkNoSelection	1	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:PNR	PersonnelConfigurationSelection	PNR		N	lkPerson	2	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:KST	PersonnelConfigurationSelection	KST		N	lkCostCenter	3	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:BER	PersonnelConfigurationSelection	BER		N	lkArea	4	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:ABT	PersonnelConfigurationSelection	ABT		N	lkDepartment	5	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:PKREIS	PersonnelConfigurationSelection	PKREIS		N	lkperson.employee_subgroup	6	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:TAETIGKEIT	PersonnelConfigurationSelection	TAETIGKEIT		N	lkperson.function	7	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:BESCHVERH	PersonnelConfigurationSelection	BESCHVERH		N	lkperson.employment_relationship	8	Local->WageTypeInteractions->ReferenceData
Local	WageTypeInteractions	PersonnelConfigurationSelection:NSTMP	PersonnelConfigurationSelection	NSTMP		N	lkperson.DoesNotClock	9	Local->WageTypeInteractions->ReferenceData

Authorizations [2]								
	Domain Set	Domain	Authorization Type	Authorization Context	Authorization Id	Authorization Key	Authorization Designation	Parer
	Local	WageTypeInteractions	RefDatItem	PersonnelConfigurationSelection	NSTMP	PersonalTimeEvaluationControlDoesNotClock	Person does not clock	Local
	Local	WageTypeInteractions	Application		WageTypeInteractions	wtia.*	Imported from menu	Local